

Game of Life

Andreu Elias Puigvert Gómez - 1702901
Jaime Amselem Herrero - 1704539

Índice

- Introducción al juego de Conway's "Game of Life".
- Análisis de las características del algoritmo.
- Herramienta propia de análisis
- Optimizaciones del programa y resultados.
- Conclusiones

Introducción al juego de Conway's “Game of Life”.

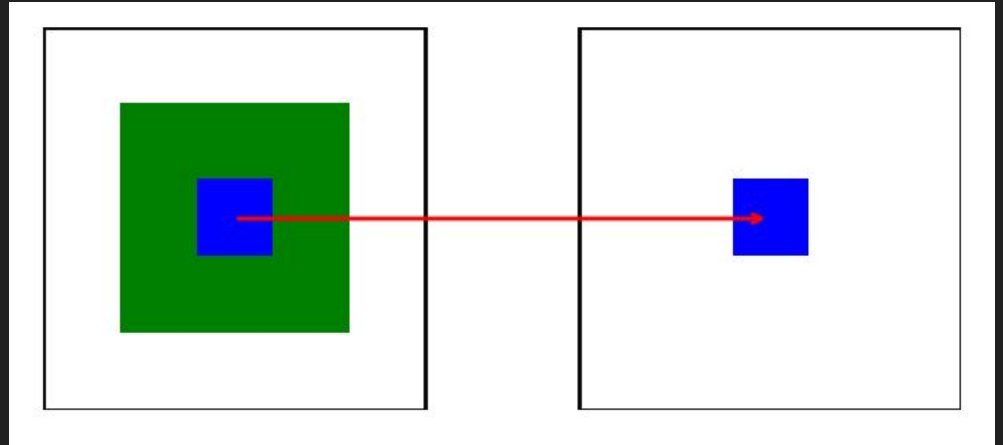
Motivación del algoritmo

John Conway



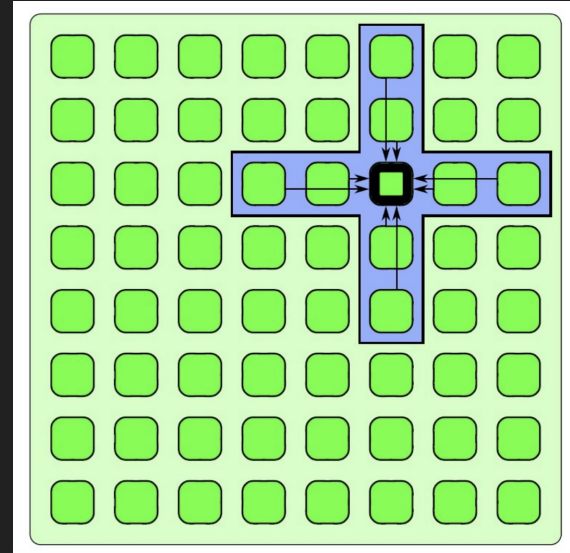
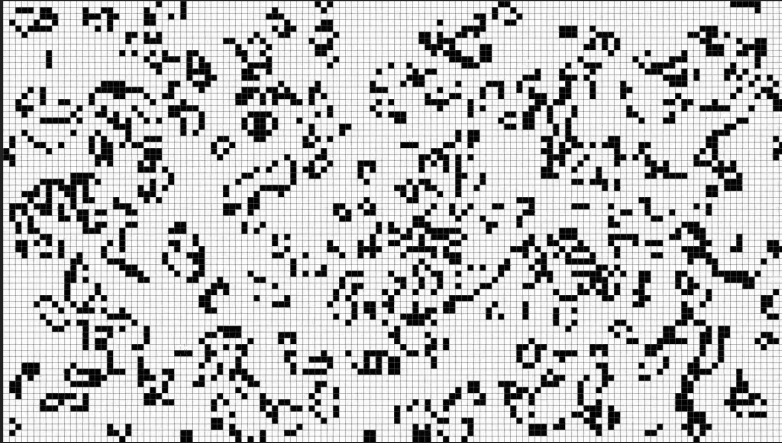
Cómo se simula la vida artificial

- Celdas que son células
- Reglas sencillas
- *Se genera el estado siguiente a partir del estado original*



Análisis de las
características del
algoritmo.

Descripción funcional STENCIL

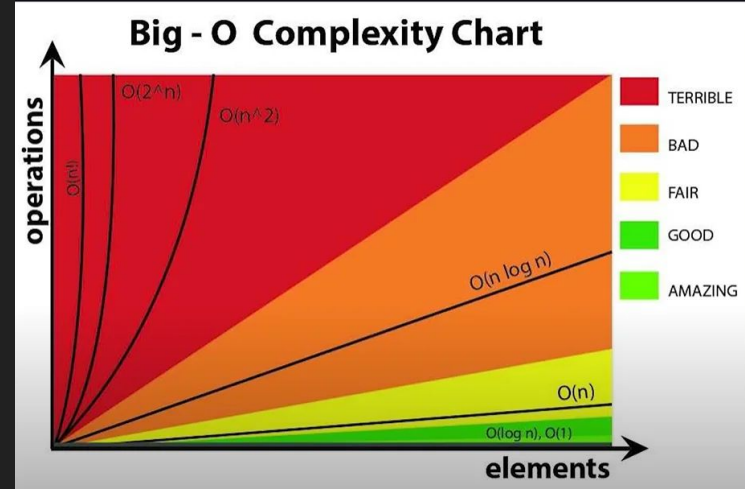


Complejidad espacial

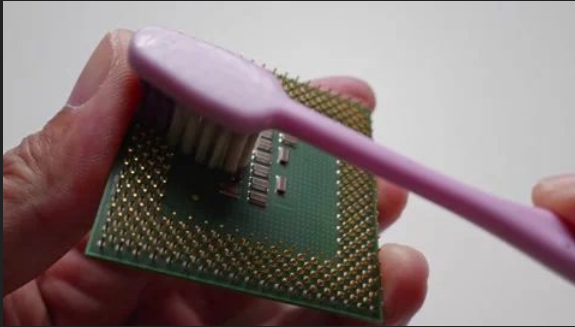
$O(N^2)$

Complejidad computacional

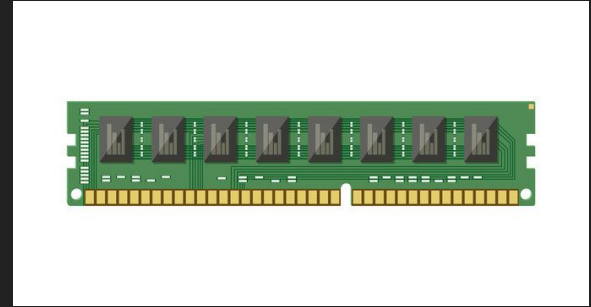
$O(N^2 \cdot n^{\circ} \text{ de iteraciones})$



Posibles cuellos de botella: Throughput-bound



Cómputo



Memoria

¿Cómo poder
analizarlo el
algoritmo?

Desarrollamos una herramienta propia para medir rendimiento y cuellos de botella.

Experimentos a probar

```
1 #!/bin/bash
2 # Aquí poner los experimentos a calcular
3 # Formato: row col iter threads name
4 experiments=(
5     "120 12000 10000 1 Entra_L1"
6     "120 320000 1000 1 Entra_L2"
7     "120 4700000 100 1 Entra_L3"
8     "120 5000000 100 1 Entra_M"
9     "12 10000000 100 1 Código_Original"
10    "12 20000000 100 1 King_Batch"
```

Por cada experimento

```
13 if [ -i "$Seed" ]; then
14     perf stat ./tmp $Iterations > results.txt
15 else
16     perf stat ./tmp $Iterations $Seed > results.txt
17 fi
18 rm ./tmp.c
19 rm ./tmp
20 # Extraer datos
21 ciclos=$(grep "cycles" results.txt | awk '{print $1}' | sed 's/,//g')
22 instructions=$(grep -E "instructions" results.txt | awk '{print $1}')
23 tiempo=$(grep "seconds time elapsed" results.txt | awk '{print $1}')
24 reloj=$(grep -E "cpu_core/cycles:u" "S" results.txt | awk -F'#' '{print $2}' | awk '{print $1}')
25 ratio=$(awk -v t="$tiempo" -v it="$Iterations" -v cel=$((rows*columns)) \
```

Genera json

```
    "porcentaje_libgomp": "0.14%"
  },
  "750x750x1x10000": {
    "ciclos": 7.595,
    "instructions": 25.733,
    "IPC": 3.38815,
    "tiempo": 1.772276266,
    "reloj": 4.288,
    "n_threads": 1,
    "iterations": 10000,
    "nameExperiment": "Entra_L2",
    "porcentajeActualizarTablero": "98.62%",
    "porcentaje_libgomp": "0.01%"
```

Generamos excel

```
1 import json
2 from openpyxl import Workbook
3 with open("result.json", "r") as f:
4     datos = json.load(f)
5
6 wb = Workbook()
7 ws = wb.active
8
9 # Escribir datos en celdas
10 ws["A1"] = "Nombre Experimento"
11 ws["B1"] = "X"
12 ws["C1"] = "Y"
```

Optimizaciones
del programa.

Resultados de optimizaciones directas

X	Y	Iteraciones	Tiempo Original	SpeedUp x 1 thread Nueva versión	SpeedUp x 6 thread Nueva versión	SpeedUp x 12 thread Nueva versión
150	150	100000	9,4	40,8x	84,1x	100,2x
750	750	10000	29,3	59,3x	189,1x	278,2x
3000	3000	1000	113,5	126,9x	742,7x	578,4x
3100	3100	1000	121,8	142,3x	696,2x	773,1x
10000	10000	100	254,0	222,3x	359,2x	348,0x
20000	20000	100	1157,1	248,6x	419,9x	388,4x
100000	1000	100	94,9	83,9x	140,3x	131,6x
200000	2000	100	596,1	130,2x	217,4x	196,8x

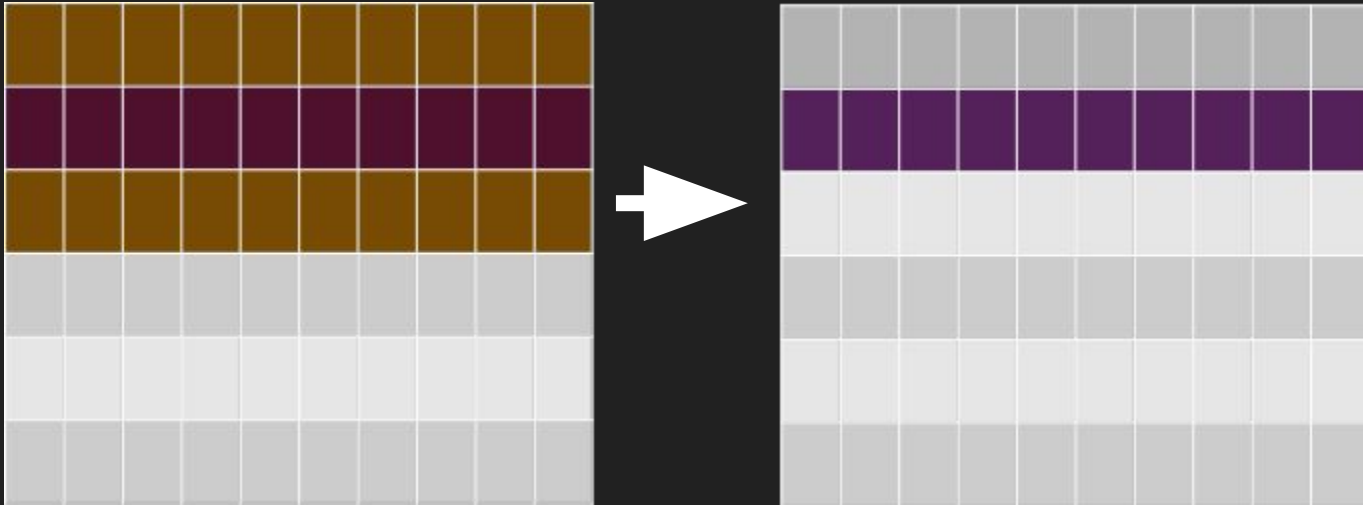
Optimizaciones directas

- Loop interchange
- Convertir los arrays a char
- Eliminar condicionales if
- Intercambiamos roles entre destino y origen
- Calculamos el checksum en la última iteración
- Paralelizar el código
- Cambiamos el cómo se inicializa el tablero



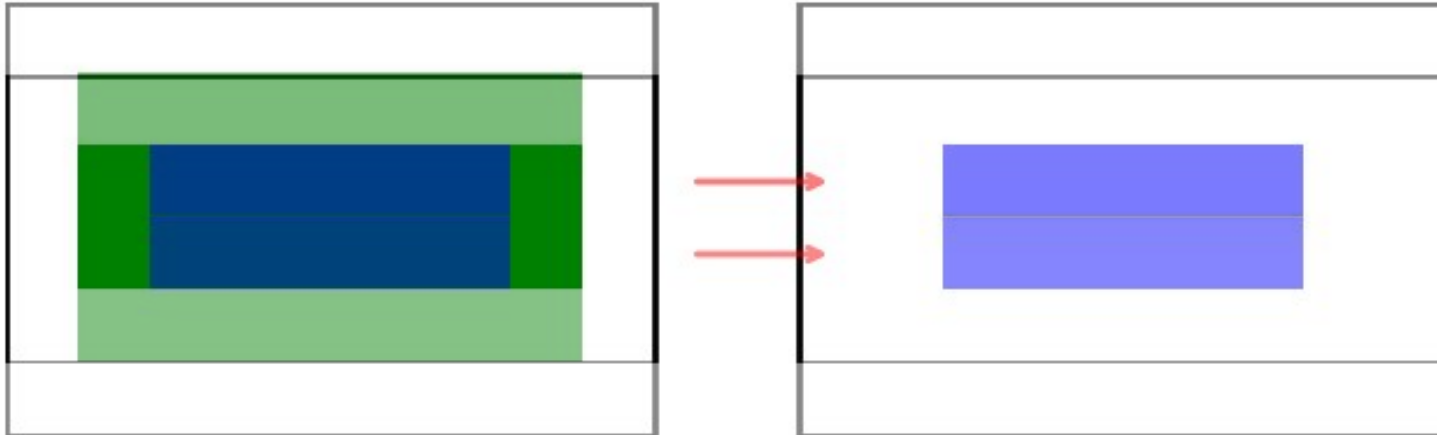
Limitaciones, problemas de BW de memoria

Si las filas son grandes: $3R + 2W$ accesos Memoria DDR4 / Por celda



Optimización 2: dividir por columnas

Si las filas son grandes: 1 R + 2 W accesos Memoria DDR4 / Por celda

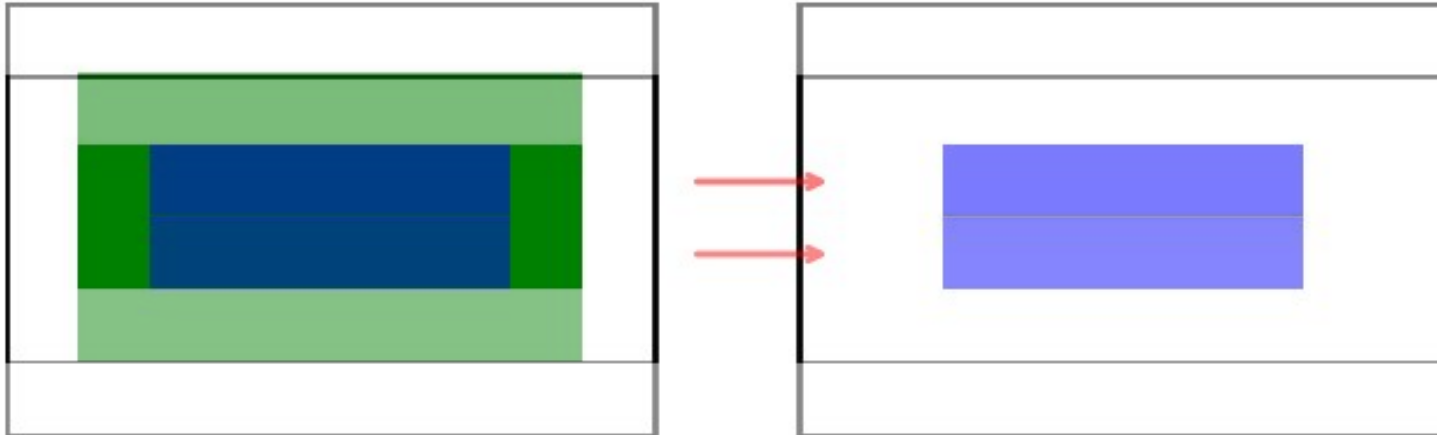


Resultados:

X	Y	Iteraciones	Tiempo Código directo	SpeedUp
150	150	100000	1,24	1,98x
750	750	10000	2,45	6,24x
3000	3000	1000	1,54	4,55x
3100	3100	1000	1,67	4,15x
10000	10000	100	2,10	2,05x
20000	20000	100	7,01	1,92x
20000	100000	100	36,93	2,09x
100000	100000	100	197,02	2,34x

Limitación, sigue siendo:

Accesos Memoria DDR4



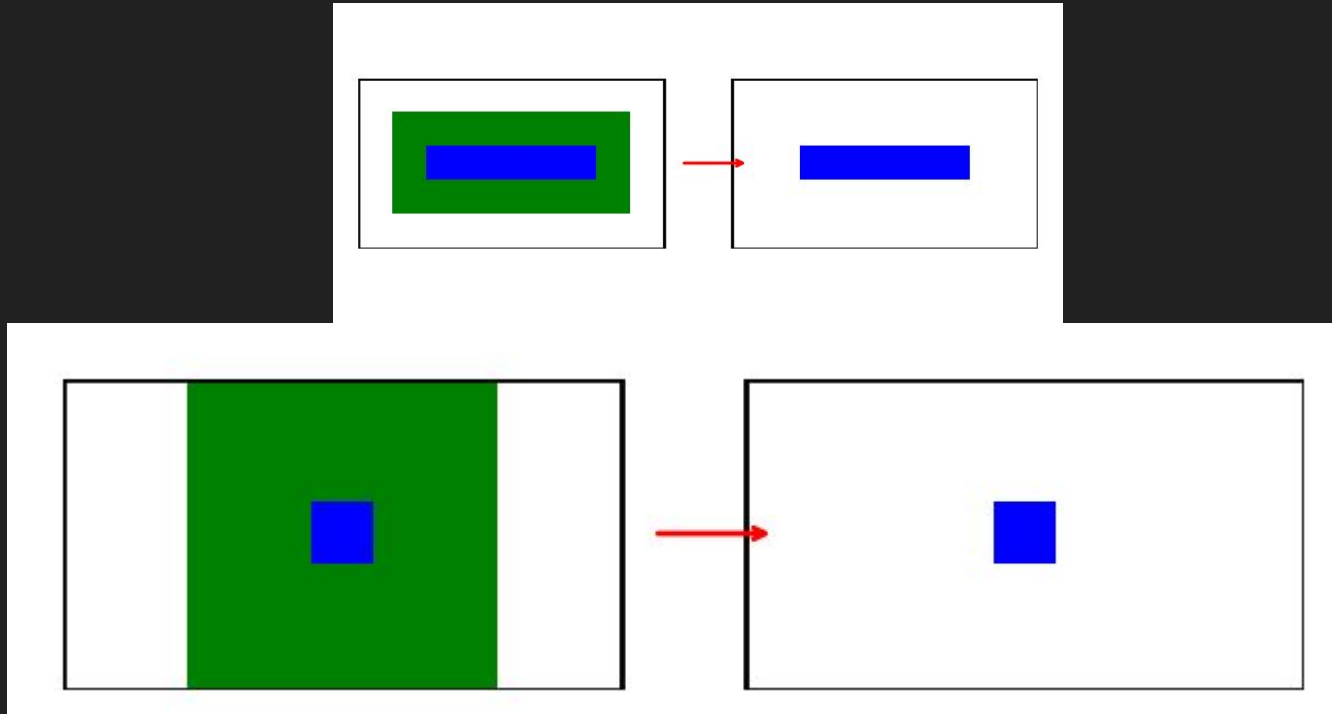
¿Posibles caminos para mejorar?

	Possible speed up para 100000x100000 it = 100 si siempre es memoria DDR4 el límite	Dificultad de vectorizar(0-10)	Dificultad de implementar
Compresión bitmap	8x (si sigue limitando el cómputo)	10	10
Tiling	1,18x-1.38x	0	9
Dos iteraciones en una	2x	5	7,5



Fuentes de los estimados en el
documento y el [github](#)

Optimización 3: calcular dos iteraciones a la vez



Resultados

X	Y	Iteraciones	Tiempo Código Columnas	SpeedUp	Hyperthreading para mejor caso
150	150	100000	0,63	1,12x	No
750	750	10000	0,39	0,56x	No
3000	3000	1000	0,34	0,63x	No
3100	3100	1000	0,40	0,64x	No
10000	10000	100	1,02	1,47x	No
20000	20000	100	3,65	1,41x	No
20000	100000	100	17,66	1,38x	No
100000	100000	100	84,36	1,24x	Sí

Problemas de implementación

Vectorización: no aplica.

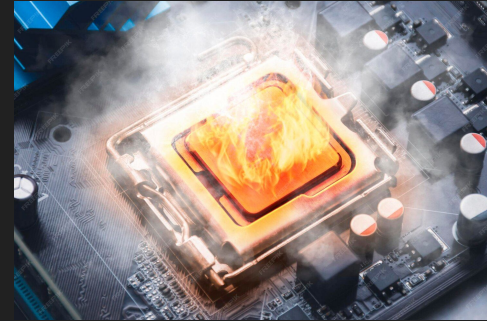
Funciones inline: no utilizadas.

8 casos especiales.



Limitaciones

- Antes, stencil-9 -> bandwidth-bound memoria
- Ahora, stencil-25 -> bandwidth-bound computo



Posibles caminos para seguir optimizando



Conclusiones

